

P1725R0

Modular Topic Design

Published Proposal, 2019-06-17

This version:

<https://wg21.link/p1725>

Author:

[Christopher Di Bella](#)

Audience:

SG20

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

SG20 advised that [\[P1389R0\]](#) should move away from a linear stage-based model into a more modular topic-based model. This proposal aims to start the discussion for evaluating the the class of modules that are suitable for curriculum guidelines.

Table of Contents

1 Change log

1.1 From P1725R0 to P1725R1

1.1.1 Changes initiated by the author

1.1.2 Changes initiated by SG20

2 Motivation

3 Design goals

4 A non-exhaustive list of modules

5 Acknowledgements

References

Informative References

§ 1. Change log

§ 1.1. From P1725R0 to P1725R1

§ 1.1.1. Changes initiated by the author

1. adds concurrent debugging
2. revises ‘[contract](#)’ to ‘[design-by-contract](#)’ to keep the theme of the topic, while distancing P1725 from the contracts language feature
3. replaces ‘[contract](#)’ (specifically in the functions module) with 'pre-conditions, post-conditions, assertions', to keep the theme of the topic, while distancing P1725 from the language feature

§ 1.1.2. Changes initiated by SG20

The following items were added to the list of modules and topics, as requested by SG20 in Cologne.

We would like to see the following added to the 'C++ Compilation Model' module.

- linkage
- phases of translation
- separate compilation
- macros
- name mangling
- constant expressions
- constant evaluation
- build systems
- heterogeneous compilation
- extern "C"

Poll result: Unanimous consent.

We would like to see the following added to the 'Error handling' module.

- C-style error-codes
- programming with sum types as error handling mechanisms (replaces ‘monadic programming’)
- `errno`
- `static_assert`, `const-model` => error when mutating
- classes of errors (compile-time, linker, run-time, logic)
- bug vs exceptional situation

Poll result: Unanimous consent.

We would like to see a module for 'meta-error handling (coping)' introduced to P1725.

Poll result: Unanimous consent.

We would like to see the following topics added to the 'meta-error handling (coping)' module.

- run-time debugging
- compile-time debugging
- coping with build error output
- parallel debugging

Poll result: Unanimous consent.

We prefer for 'meta-error handling (coping)' to be adopted by the 'tools' module.

Poll result:

SF	F	N	A	SA
2	1	3	2	3

We would like to see a 'compile-time programming' module adopted into P1725.

Poll result: Unanimous consent.

We would like to see the following topics added to the 'compile-time programming' module.

- `constexpr`-programming (includes `constexpr`)
- `constexpr`-programming
- template meta-programming
- preprocessor meta-programming
- `static_assert`, `SFINAE`, *requires-expressions*

Poll result: Unanimous consent.

We encourage the author to continue working on P1725.

Poll result: Unanimous consent.

§ 2. Motivation

It was noted in the SG20 Kona face-to-face that learning is an iterative process, requiring malleability, and that the stages outlined in P1389 were too rigid and linear; this might be too restrictive on curriculum design as a result. This proposal sets out to ask SG20 to discuss the ideas behind a set of topic ‘modules’ where stages are embedded into a module. The benefit of the guidelines having sets of topics as opposed to sets of stages is that teachers will be able to formulate their own curricula more easily.

P1389R0 stated that that a student should be able to meet all of the objectives and satisfy all of the outcomes in Stage 1 before they are ready to progress to Stage 2, and so on. This means that there are some unrelated topics that are coupled together. In hindsight, the author of P1389 also acknowledges that the sets of stages is closer to a prescribed curriculum than intended. For example, some of P1389’s Stage 1 has algorithms, basic I/O, computation constructs, containers, contracts, and designing and using classes. None of these topics are strictly related, but Stage 2 is completely blocked until all the topics in Stage 1 are exhausted. Since P1389 advises derived types be introduced in Stage 2 -- and Stage 2 is blocked until Stage 1 is completed -- P1389 excludes the possibility of a curriculum introducing derived types as a part of the class design topic, despite it has no real dependency on basic I/O.

If P1389 or its successor were to advise modules of topics, where strict dependencies are outlined, then this would make it possible for both curriculum designs in the previous paragraph to co-exist and still conform to the recommendations outlined in the 'Standing Document for Teaching C++ to C++ Novices with Programming Experience' (P1389’s target). This will help shape the Standing Documents that SG20 outputs, so that they are significantly less prescriptive, and satisfy the advisory role outlined in the SG20 charter, as opposed to accidentally becoming a prescriptive curriculum.

§ 3. Design goals

It should be noted that while P1725 suggests a handful of modules and topics that fall within those modules, it is by no means an exhaustive list of either. Furthermore, P1725R0 does not make any attempt to draw a dependency graph among the topics listed in each topic. The author is requesting the help of SG20 to formulate an exhaustive list of modules and topics, as well as a strong dependency graph that will help teachers structure their curricula.

These modules are designed to have their own associated objectives and outcomes. A curriculum’s objectives are essentially a contract between a student and the course. By the end of a course, a student should have developed certain knowledge, understanding, and skills about particular topics. These are assessable through course outcomes, which advise that a student should be able to do something

related with that knowledge, or related to that skill. For example, Table 1 outlines a few objectives and outcomes from a hypothetical introductory C++ course.

Note: Table 1 is an example set of objectives and outcomes for a particular hypothetical course and is deliberately incomplete: P1725 is *not* proposing the contents of Table 1 be added to any document published by SG20.

Objective	Outcome
Students will develop	A student
1. knowledge and understanding about how software engineers build systems	P1.1 chooses the correct data structures, algorithms, libraries, and tools for engineering software P1.2 consults the most appropriate resources for building a system
2. knowledge and understanding about why testing is an important for software development	P2.1 designs unit-tests to validate that the individual components of their are correct P2.2 designs integration-tests to validate that their individual components work together to provide the correct solution
3. knowledge and understanding about why benchmarking is important for software development	P3.1 designs benchmarks to prove that one choice is "more efficient" than another
4. knowledge and understanding about software engineering tools	P4.1 chooses the correct tools to help solve the problem
5. skills in the C++ programming language	P5.1 wirtes programs that correctly sovlve the problem using C++ P5.2 writes programs that take advantage of modern C++ programming techniques

P5.3

reads and understands existing C++ code to understand the current state of a solution

P5.4

modifies or extends existing C++ code without the need for major rewrites to well-written code

§ 4. A non-exhaustive list of modules

Table 2 outlines a non-exhaustive list of modules and their related topics. The modules are not listed in a meaningful order; instead, they are listed so that the table is evenly formatted.

C++ compilation model	Error handling	Meta-error handling
• declarations • definitions • headers • modules • translation units • linkage • phases of translation • separate compilation • macros • name mangling • constant expressions • constant evaluation • build systems • heterogeneous compilation • <code>extern "C"</code>	• design-by-contract • exception handling • sum types as error handling • C-style error-codes • <code>errno</code> • <code>static_assert</code> • classes of errors (e.g. compile-time, linker, run-time, logic) • bug vs exceptional situation	• Run-time debugging • Compile-time debugging • Coping with build errors • Parallel debugging • Concurrent debugging
Compile-time programming	I/O	
• <code>constexpr</code>-programming (includes <code>constexpr</code>)	• character I/O • from-string conversions	

- `constexpr`-programming
- template meta-programming
- preprocessor meta-programming
- `static_assert`, SFINAE, *requires-expressions*
- file I/O
- GUIs (using a third-party library)
- to-string conversions

C++ object model

- copy semantics
- constant expressions
- constant objects
- constructors
- conversions
- destructors
- free store
- lambdas
- move semantics
- objects
- pass-by-value
- pass-by-reference
- pointers
- references
- references-to-`const`
- reference semantics
- resources
- rule-of-five
- rule-of-zero
- scope
- smart pointers
- special member functions

Functions

- calling functions
- pre-conditions, post-conditions, assertions
- function templates
- lambdas
- member functions
- overloading
- parameters
- pass-by-value
- pass-by-reference
- returning multiple values
- return value optimisation
- user-defined literals
- virtual functions

User-defined types

- access
- class templates
- constructors
- data members
- derived types
- destructors
- friends
- interface inheritance
- member functions
- `optional` and `variant` as sum types
- rule-of-five
- rule-of-zero
- special member functions
- static members
- `structs` and `classes` as product types
- user-defined literals
- variable templates
- virtual functions

• types • values • value semantics		
Program design • algorithms • concepts • containers • design-by-contract • contributing to a pre-existing project • error handling • interface inheritance • I/O design • iterators • ranges • resource handling	Type system • conversions • fundamental types • function overloading • scoped enumerations • template type parameters • user-defined types	Tooling • build system • compiler • continuous integration • debugger • formatter • IDE • libraries • linter • optimiser • package manager • test tools • version control
History of C++ No topic suggestions at present.	Interop with C No topic suggestions at present.	Undefined behaviour No topic suggestions at present.

An astute reader should notice that there are a few topics that are mentioned in multiple modules. This highlights that the above modules could perhaps be further refined *or* that certain topics cannot be taught in isolation. Given that features of C++ are designed to work with one another, the author believes that the latter case is more likely. The reader should also note that both ‘types’ and ‘user-defined types’ have been listed as modules *and* topics within other modules.

The author would like to propose that SG20 adopt the module/topic model -- even if it is in a different form to what is outlined above -- and begin discussing the kinds of modules desired, the topics underneath each module, the objectives for each module, and the relevant module outcomes associated with each objective.

§ 5. Acknowledgements

The author would like to thank JC van Winkel for his direct feedback on this proposal, as well as the members of SG20 who provided feedback on P1389R0's structure.

§ References

§ Informative References

[P1389R0]

Christopher Di Bella, Simon Brand, Michael Adams. [Standing Document for SG20: Guidelines for Teaching C++ to Beginners](https://wg21.link/p1389r0). URL: <https://wg21.link/p1389r0>